

Continuum Simulator: monitoraggio ambientale e aggregazione distribuita Edge–Cloud

Alessandro Vassallo – Matricola 0366572
Università degli Studi di Roma Tor Vergata
Sistemi Distribuiti e Cloud Computing – A.A. 2025/26

Sommario—Questa relazione presenta un’applicazione distribuita sviluppata in Go per il monitoraggio ambientale nel compute continuum. Il sistema riproduce misure reali derivate da Sensor.Community e distribuisce l’elaborazione tra livelli Edge e Cloud mediante MQTT e Apache Kafka. La soluzione adotta un’aggregazione gerarchica basata su finestre temporali e un livello Cloud scalabile orizzontalmente tramite partizionamento Kafka e consumer group. La valutazione sperimentale analizza il comportamento del sistema al variare del numero di Cloud Worker e delle condizioni di comunicazione.

I. INTRODUZIONE

L’obiettivo del progetto è lo sviluppo di un’applicazione distribuita nel compute continuum per il monitoraggio di misure ambientali, distribuendo l’elaborazione tra livelli Edge e Cloud. Il caso di studio utilizza dati provenienti da sensori BME280 del dataset pubblico Sensor.Community, relativi a temperatura, umidità e pressione e raccolti nel mese di gennaio 2025.

Il dataset storico viene utilizzato per simulare un sistema di monitoraggio continuo. Le osservazioni vengono riprodotte preservandone le distanze temporali in event time mediante un replay accelerato e inviate ai rispettivi Edge Site. Ciascun Edge esegue una prima fase di validazione e aggregazione locale, mentre il livello Cloud combina successivamente i risultati provenienti dalle diverse zone per ottenere una vista complessiva del sistema. Questa suddivisione permette di distribuire la computazione lungo il continuum e di ridurre la quantità di dati che deve essere trasferita verso il Cloud rispetto all’invio di ogni singola osservazione.

Poiché il progetto è svolto individualmente, tra i requisiti alternativi previsti dalla traccia è stata scelta la scalabilità orizzontale. La valutazione sperimentale studia quindi il comportamento del sistema al variare del numero di Cloud Worker, mantenendo invariato il workload, e analizza inoltre l’impatto delle condizioni di rete sulla pipeline distribuita. L’applicazione è sviluppata interamente in Go, containerizzata tramite Docker e distribuita su AWS mediante un processo di deployment automatizzato.

II. DATASET E COSTRUZIONE DEL WORKLOAD

Lo scenario è stato costruito a partire dal dataset pubblico Sensor.Community relativo ai sensori ambientali BME280. È stata considerata la finestra temporale di gennaio 2025 e il dataset, di dimensione pari a circa 6.36 GB, è stato analizzato in modo incrementale per evitare il caricamento completo in memoria.



Figura 1. Topologia geografica della popolazione europea e centroidi dei 13 Edge Site.

Una prima fase di preprocessing ha permesso di verificare la qualità dei dati, filtrare le osservazioni esterne alla finestra temporale e individuare i sensori dotati di coordinate geografiche valide. Delle variabili disponibili, le coordinate geografiche sono state utilizzate per la costruzione della topologia, mentre temperatura, umidità e pressione sono state mantenute come misure da riprodurre durante l’esecuzione. Le variabili relative all’altitudine e alla pressione al livello del mare, completamente prive di valori, sono state escluse.

Lo studio è stato limitato all’area europea. Per ogni sensore è stata determinata una posizione rappresentativa e le coordinate geografiche WGS84 sono state proiettate nel sistema metrico EPSG:3035, in modo da applicare il clustering su distanze spaziali coerenti. La popolazione risultante comprende 5102 sensori europei localizzabili.

La suddivisione in Edge Site è stata ottenuta tramite K-Means sulle coordinate proiettate. Sono stati valutati valori di k compresi tra 5 e 15 utilizzando il Silhouette Score come criterio principale di selezione, affiancato dall’inertia e dal raggio massimo dei cluster come indicatori diagnostici. La configurazione selezionata è risultata $k = 13$, con Silhouette Score pari a circa 0.521; l’elbow method individua invece $k = 9$, utilizzato solamente come riferimento diagnostico.

Una volta determinata la topologia sull’intera popolazione europea, è stato costruito il workload sperimentale selezionando 150 sensori. I sensori sono stati distribuiti nel modo più

uniforme possibile tra i 13 Edge Site (11–12 sensori per Edge). La selezione all'interno di ciascun Edge è stata effettuata senza sostituzione utilizzando un seed pseudo-casuale fissato. In questo modo ogni Edge partecipa in maniera comparabile agli esperimenti, pur mantenendo le differenze naturali nel tasso di generazione degli eventi.

Infine, la fase di preprocessing produce due tipi di artefatti utilizzati durante l'esecuzione: il file rappresentante la topologia, che associa i sensori selezionati agli Edge Site, e 13 shard di replay, uno per ciascun Edge, contenenti identificativo del sensore, timestamp, pressione, temperatura e umidità.

III. ARCHITETTURA DEL SISTEMA

L'architettura realizzata è una pipeline distribuita nel compute continuum, organizzata tra livelli Edge e Cloud come mostrato in Fig. ?? . Il sistema riproduce le osservazioni dei sensori selezionati dal dataset storico, le elabora localmente negli Edge Site e propaga verso il Cloud risultati progressivamente aggregati.

La pipeline è composta dai seguenti stadi logici.

- 1) **Simulator:** Per ciascuno dei 13 Edge Site è presente un Simulator che riproduce gli eventi dei sensori assegnati preservandone le distanze temporali in event time, opportunamente accelerate. La telemetria viene pubblicata tramite MQTT verso il broker Mosquitto associato al sito.
- 2) **Elaborazione Edge:** Ogni Edge Site comprende un broker MQTT e un Edge Gateway. Il gateway riceve e valida le misure di temperatura, umidità e pressione e le aggrega localmente su finestre temporali di configurabili. Questa prima elaborazione consente di ridurre il volume di dati inoltrato verso il Cloud, trasferendo statistiche aggregate anziché le singole osservazioni.
- 3) **Trasporto Edge-Cloud:** Gli aggregati prodotti dagli Edge vengono pubblicati sul topic Kafka edge-aggregates. Kafka è utilizzato per disaccoppiare produttori e consumatori e per fornire persistenza e ordinamento dei record all'interno delle partizioni [3]. Gli Edge sono distribuiti staticamente tra un numero configurabile di partizioni, bilanciando il carico atteso sulla base del numero di eventi presenti nei rispettivi replay shard.
- 4) **Elaborazione Cloud.** I Cloud Worker appartengono allo stesso consumer group Kafka. Ciascuna partizione è assegnata a un solo worker e costituisce un'unità indipendente di aggregazione. I contributi provenienti dagli Edge vengono combinati su finestre temporali di durata configurabile, definita come multiplo della finestra Edge. Aumentando il numero di worker è quindi possibile elaborare più partizioni in parallelo, fino al limite imposto dal numero di partizioni del topic.
- 5) **Aggregazione globale e persistenza:** I risultati dei Cloud Worker vengono pubblicati sul topic cloud-partition-aggregates e consumati dal Global Aggregator. Quest'ultimo combina i contributi relativi allo stesso intervallo temporale provenienti dalle

diverse partizioni e produce un unico risultato globale, successivamente scritto su log oppure persistito in PostgreSQL.

Il numero di Cloud Worker può essere modificato tra esecuzioni senza cambiare la suddivisione logica del carico né il significato dei risultati prodotti. Questo permette di utilizzare il parallelismo del livello Cloud come principale parametro nella valutazione della scalabilità orizzontale.

IV. IMPLEMENTAZIONE

A. Aggregazione Edge e Cloud

L'elaborazione utilizza finestre tumbling basate sull'event time. Ciascun Edge mantiene una finestra corrente e assegna ogni evento alla finestra determinata dal relativo timestamp. Per ogni grandezza ambientale vengono mantenuti il numero di valori validi e non validi, la somma, il minimo e il massimo. Al momento della chiusura della finestra viene inoltre calcolata la media sui soli valori validi.

Le statistiche sono state scelte in modo da poter essere combinate nei livelli successivi senza conservare le singole osservazioni. Indicando con n_j il numero di valori validi e con s_j la loro somma nel contributo j , la combinazione di più aggregati utilizza

$$n = \sum_j n_j, \quad s = \sum_j s_j, \quad \bar{x} = \frac{s}{n}. \quad (1)$$

Minimo e massimo globali sono ottenuti rispettivamente come minimo e massimo dei contributi che contengono almeno un valore valido. In questo modo la media globale non viene calcolata come media delle medie locali, che produrrebbe un risultato errato quando gli aggregati contengono numerosità differenti.

Le finestre Cloud hanno durata configurabile, ma devono essere un multiplo intero della durata delle finestre Edge. Ogni EdgeAggregate appartiene quindi interamente a una sola finestra Cloud. I Cloud Worker combinano gli aggregati ricevuti per partizione Kafka e producono un CloudPartitionAggregate; il Global Aggregator applica infine la stessa operazione di riduzione ai contributi delle diverse partizioni relativi allo stesso intervallo temporale.

B. Progresso event-time e completamento

La chiusura delle finestre Cloud è governata dal progresso in event time delle sorgenti e non da timeout in processing time. Ogni EdgeAggregate contiene il campo CompleteThrough, che certifica che l'Edge non produrrà successivamente aggregati con fine della finestra precedente o uguale a tale istante.

Quando l'arrivo di un evento determina il passaggio a una nuova finestra, l'Edge emette l'aggregato della finestra precedente e può avanzare CompleteThrough fino all'inizio della nuova finestra. In questo modo vengono certificate anche eventuali finestre intermedie prive di eventi. Gli Edge non producono infatti aggregati espliciti per le finestre vuote.

Il Cloud mantiene separatamente il progresso di ciascun Edge appartenente alla partizione Kafka elaborata. Indicando

con C_e il valore `CompleteThrough` dell'Edge attivo e e con S il massimo disallineamento ammesso tra le sorgenti, la frontiera della partizione è determinata da

$$W_P = \max \left(\min_{e \in E_P} C_e, \max_{e \in E_P} C_e - S \right). \quad (2)$$

Finché almeno un Edge attivo non ha ancora comunicato il proprio progresso, la frontiera non può avanzare. In condizioni normali essa coincide quindi con il minimo progresso degli Edge della partizione; il secondo termine impedisce invece che una sorgente eccessivamente in ritardo blocchi indefinitamente la finalizzazione delle finestre. Una finestra Cloud viene emessa quando il proprio estremo superiore non è successivo a W_P .

Un aggregato relativo a una finestra già superata dalla frontiera è considerato *late*: non modifica né riapre il risultato già emesso. Il progresso contenuto nel record può comunque essere utilizzato per aggiornare lo stato dell'Edge e consentirgli di riallinearsi alle altre sorgenti.

Poiché il workload è ottenuto dal replay di un dataset finito, è inoltre necessario determinare esplicitamente la conclusione del flusso. Al termine del proprio replay ciascun Simulator invia un marker di fine input al rispettivo Edge. L'Edge pubblica l'ultima finestra ancora aperta e, successivamente, un `EdgeEndOfInput`. Quando tutti gli Edge appartenenti a una partizione sono terminati, il Cloud Worker emette gli eventuali aggregati ancora pendenti e infine un `PartitionEndOfReplay`.

Il Global Aggregator utilizza i risultati delle partizioni insieme ai relativi progressi e marker terminali per distinguere un contributo non ancora arrivato da un contributo effettivamente assente. Una finestra globale può quindi essere prodotta quando, per ogni partizione sorgente, è disponibile il relativo aggregato oppure è stato certificato che non ne verranno prodotti altri per quell'intervallo. Il replay termina quando tutte le partizioni hanno raggiunto lo stato terminale e non rimangono finestre pendenti.

C. Comunicazione e affidabilità

La comunicazione tra Simulator ed Edge utilizza MQTT. Gli eventi dei sensori sono serializzati in JSON e pubblicati con QoS 0. JSON è stato scelto in questo tratto della pipeline per la semplicità della rappresentazione e per la facilità con cui i messaggi possono essere ispezionati durante lo sviluppo e il debugging. La telemetria viene inoltre prodotta con frequenza elevata: l'utilizzo di QoS 0 evita di attendere una conferma per ogni singola osservazione e permette al replay di procedere secondo il rate configurato. Il marker di fine replay utilizza invece QoS 1 e viene considerato completato soltanto dopo la ricezione del relativo PUBACK, poiché la sua perdita impedirebbe di determinare correttamente la terminazione della sorgente.

La comunicazione tra Edge e Cloud e tra Cloud Worker e Global Aggregator è invece affidata ad Apache Kafka. In questo tratto i messaggi sono serializzati in formato Apache Avro. Rispetto al JSON, Avro fornisce una rappresentazione binaria

più compatta e definisce esplicitamente la struttura e i tipi dei record scambiati, caratteristica utile per aggregati composti da numerosi campi numerici e temporali. Gli schemi Avro sono inclusi direttamente nell'applicazione e devono quindi essere condivisi in modo coerente da producer e consumer; non viene utilizzato uno Schema Registry.

Il tipo logico di ciascun record Kafka è indicato mediante l'header `record_type`. In questo modo lo stesso meccanismo di trasporto può veicolare aggregati, informazioni di progresso e marker terminali. Questi ultimi non richiedono dati applicativi e vengono pertanto inviati con payload vuoto.

Gli Edge utilizzano writer Kafka sincroni con acknowledgment del broker. Gli aggregati appartenenti allo stesso Edge e il relativo `EdgeEndOfInput` sono scritti sulla medesima partizione; prima di pubblicare il marker terminale vengono completate le scritture degli aggregati precedenti. Analogamente, il Cloud pubblica per ciascuna partizione prima gli eventuali `CloudPartitionAggregate`, quindi il relativo `PartitionProgress` e infine, quando necessario, `PartitionEndOfReplay`. L'ordinamento permette ai consumer downstream di interpretare un'informazione di progresso solo dopo aver ricevuto tutti i dati che essa certifica.

I Cloud Worker effettuano il commit degli offset Kafka solamente dopo che il record di input è stato elaborato e gli eventuali output sono stati pubblicati con successo. I commit possono essere raggruppati in batch configurabili per ridurne il costo; la ricezione di un marker terminale forza invece il commit degli offset ancora pendenti. Il Global Aggregator applica lo stesso principio, effettuando il commit dopo l'elaborazione e la scrittura del risultato nel sink.

Questa strategia evita di confermare un record prima che i suoi effetti siano stati propagati, ma non realizza una transazione atomica end-to-end tra stato, output Kafka e offset di consumo. Un arresto dopo la pubblicazione dell'output ma prima del commit può quindi causare la riletture di un record. Per questo motivo gli aggregati utilizzano identificativi deterministici e i componenti riconoscono i duplicati identici, mentre nel sink PostgreSQL l'identificativo dell'aggregato è una chiave univoca.

D. Deployment e piattaforma software

L'applicazione è sviluppata in Go e ciascun componente principale viene distribuito come container Docker. La configurazione del sistema è descritta tramite file YAML, dai quali il programma `deploygen` genera automaticamente i file Docker Compose necessari all'esecuzione. In questo modo parametri quali numero di Cloud Worker, numero di partizioni Kafka, dimensione delle finestre e parametri di batching possono essere modificati senza intervenire sul codice applicativo.

Per il deployment distribuito l'infrastruttura AWS è definita mediante Terraform. Sono previsti quattro host EC2 con ruoli distinti: Simulator, Edge, Cloud Core e Workers. L'host Simulator esegue i processi di replay, l'host Edge ospita i broker Mosquitto e gli Edge Gateway, l'host Workers esegue le repliche dei Cloud Worker, mentre Cloud Core ospita

Kafka e il Global Aggregator. La separazione consente di far attraversare realmente la rete ai flussi Simulator–Edge ed Edge–Cloud e di modificare indipendentemente il numero di worker utilizzato negli esperimenti.

La persistenza dei risultati globali può essere affidata a un’istanza PostgreSQL gestita tramite Amazon RDS. Il database non è esposto pubblicamente ed è raggiungibile dal solo Cloud Core. L’infrastruttura, la configurazione applicativa e l’avvio degli esperimenti sono gestiti tramite script dedicati, in modo da rendere ripetibile la preparazione dell’ambiente e la raccolta degli artefatti prodotti da ogni esecuzione.

Le principali librerie utilizzate dall’applicazione sono Eclipse Paho per MQTT, *kafka-go* per la comunicazione con Kafka, *goavro* per la serializzazione Avro e *pgx* per l’accesso a PostgreSQL. Il preprocessing offline del dataset è invece realizzato in Python mediante *pandas*, *scikit-learn* e *pyproj*. Non viene utilizzato un framework esterno di stream processing: la gestione delle finestre, dello stato e del progresso event-time è implementata direttamente nei componenti Go.

V. VERIFICHE E VALUTAZIONE SPERIMENTALE

A. Verifiche di correttezza

Le suite Go verificano codec Avro, attesa degli Edge lenti, partizioni vuote, progresso, EOS, duplicati e conflitti. Controllano inoltre l’ordine pubblicazione–commit e l’equivalenza della computazione con $W = 1, 2, 4, 6$. Il confronto riguarda gli aggregati e le statistiche prodotti, non il tempo di esecuzione. La suite ordinaria `go test ./... -count=1`, rieseguita l’11 settembre 2026 durante la preparazione della bozza, è stata superata. I programmi Simulator ed Edge non hanno test Go propri nella versione ispezionata: la copertura automatizzata non va estesa per inferenza a tutta la pipeline.

Un test opt-in utilizza broker Kafka reali per esercitare consumer group e i due passaggi sul log; la verifica registrata nel documento di contratto del 9 settembre riporta otto casi superati, con tredici Edge oppure un solo Edge e cinque partizioni vuote. Tale prova non è stata rieseguita per redigere questa bozza e non sostituisce un benchmark completo MQTT–Edge–Cloud. Gli script di deployment includono inoltre test Python per parser, configurazioni, quote e controlli di qualità. Le integrazioni opzionali richiedono dipendenze esplicitamente abilitate; gli esiti e gli eventuali test saltati devono accompagnare la verifica finale.

B. Evidenze preliminari disponibili

Nel workspace sono conservate due prove AWS, datate 9 e 10 settembre 2026, entrambe con un worker e accelerazione $A = 10.000$. I riepiloghi riportano, in ciascuna prova, 2.106.784 eventi offerti, elaborati all’Edge e rappresentati nei risultati globali. Sono presenti 105.608 aggregati Edge e 2.710 finestre globali; i contatori di drop e di finestre globali incomplete sono nulli.

Tuttavia entrambe le esecuzioni hanno `status=failed` e un esito negativo dei controlli di qualità (`quality_status=fail`). Il motivo registrato è `orchestration_failed`. Nella prova del 10 settembre

il log di orchestrazione segnala Kafka `unhealthy` nel controllo conclusivo. L’uguaglianza dei conteggi è un’evidenza diagnostica utile, ma non annulla il fallimento del protocollo di validazione. Queste prove non sono considerate run valide della campagna; non vengono usate per calcolare speedup o sostenere conclusioni sulla scalabilità.

Le due prove usano la strumentazione antecedente all’attuale collector Go del lag. Il costo della raccolta e la cadenza di campionamento rendono inappropriato un confronto diretto delle risorse del broker fra vecchie e nuove run. La campagna deve quindi essere ripetuta con release e strumentazione uniformi. Al momento della bozza non è disponibile una serie AWS valida e ripetuta con tutti i valori di W né una valutazione con `tc-netem`.

C. Disegno della campagna da completare

Il primo passo è calibrare A in modo che Simulator ed Edge sostengano il carico senza perdite e senza accumulare un ritardo di generazione crescente. La baseline locale usa $A = 20.000$, i file di confronto Cloud $A = 40.000$ e il pilota AWS $A = 10.000$: sono parametri configurati, non capacità misurate. Il carico definitivo deve essere fissato dopo la calibrazione su EC2.

Il confronto principale prevede $W = 1, 2, 4$, con $W = 6$ come punto aggiuntivo. Per ogni valore si mantengono uguali dataset, finestre, code, immagini, quote per replica, numero di partizioni e strumentazione. Si eseguono più livelli di carico e almeno cinque ripetizioni per configurazione, alternando l’ordine dei valori di W . Si riportano mediana e dispersione, con intervalli di confidenza ove il numero di osservazioni lo consenta. Il replay finito comprende i transitori di avvio e svuotamento: essi devono essere dichiarati, ed eventualmente analizzati anche separatamente.

La prova a carico costante isola l’effetto delle repliche su tempo di completamento e backlog. Per dimostrare il requisito della traccia occorre anche aumentare il carico e verificare se più repliche mantengano comparabili i tempi di risposta. Il carico dei worker è il rate degli aggregati Edge, non soltanto quello degli eventi grezzi. A finestre fisse, aggiungere sensori a una zona già attiva può aumentare il lavoro Edge senza aumentare il numero di record Cloud. L’accelerazione del replay aumenta invece anche la frequenza di chiusura delle finestre rispetto al tempo reale.

La rete deve essere valutata introducendo separatamente ritardo e limiti di banda nei collegamenti Simulator–Edge ed Edge–Cloud/Kafka, poi combinandoli in scenari controllati. I valori dei profili, le interfacce e le direzioni interessate dovranno essere registrati negli artefatti. Questa estensione `tc-netem` non è ancora implementata; nessun dato di questa bozza ne presuppone l’esecuzione.

D. Metriche e criteri di accettazione

Si distinguono eventi offerti, accettati nelle code, ricevuti, elaborati e inclusi nei risultati. Una run del confronto senza perdite deve soddisfare $N_{\text{offerti}} = N_{\text{Edge}} = N_{\text{Global}}$, avere EOS completi, offset finali allineati, assenza di errori di

protocollo, restart e OOM, e superare i controlli del runner. I confronti fra repliche devono verificare anche le statistiche per finestra, perché i soli conteggi non dimostrano l'equivalenza numerica dei risultati. Le run scartate vanno comunque censite.

Il throughput utile end-to-end è il numero di eventi rappresentati nei risultati diviso per la durata dichiarata della misura. Il throughput Cloud va espresso anche in aggregati Edge/s. Per un workload fisso, lo speedup può essere espresso come $S_W = T_1/T_W$ e l'efficienza come $E_W = S_W/W$, specificando che T_W include il pacing quando si usa la durata end-to-end. Una sorgente che impone il rate può rendere simili i tempi di completamento anche se la capacità interna aumenta; per questo si osservano anche lag e ritardi.

Il lag Kafka è una differenza di offset, quindi un numero di record, non una latenza in secondi. I topic contengono anche controlli di progresso ed EOS. Il suo massimo è campionato e può sottostimare il picco; le query fallite o incomplete non devono essere sostituite con zero. CPU e RAM vanno riportate per ruolo e come totale delle repliche. Sugli host T3 occorre conservare anche le metriche dei crediti CPU e la modalità di funzionamento, per non attribuire al parallelismo effetti dovuti al throttling.

Per i tempi di disponibilità delle finestre, sottrarre il timestamp storico `window_end` da `emitted_at` produrrebbe un intervallo privo di significato prestazionale. Una metrica da introdurre nella campagna usa invece il fine-finestra riportato sulla timeline accelerata:

$$D(w) = T_{\text{emit}}(w) - \left(T_0 + \frac{t_{\text{end}}(w) - t_0}{A} \right). \quad (3)$$

Essa misura il ritardo rispetto alla chiusura nominale e include attesa delle sorgenti, code e servizio; non è la sola latenza di rete né la latenza di commit del sink. Richiede orologi sincronizzati. La finestra finale chiusa da EOS va analizzata separatamente, perché può terminare prima del suo confine nominale. Il collector corrente conserva i timestamp originali, ma non calcola questa metrica: i suoi percentili non sono ancora risultati disponibili.

VI. LIMITAZIONI E DISCUSSIONE

A. Persistenza e guasti

La persistenza del log Kafka non implica il ripristino dello stato delle finestre. Worker e Global mantengono gli aggregati in RAM; i Worker committano in batch soltanto offset già elaborati, ma non esiste un checkpoint coordinato fra offset e stato. Un crash può quindi perdere contributi già consumati e committati, che non verrebbero riletti al riavvio. Non è garantita una semantica exactly-once end-to-end. Il sink PostgreSQL finale non è uno state store degli aggregatori.

Per questo motivo un rebalance dopo l'inizio dell'elaborazione invalida la run; il worker rifiuta inoltre la ripartenza da offset già committati. Le repliche devono essere pronte prima del replay e restare stabili fino al termine. Un errore richiede una nuova esecuzione completa con topic e offset puliti. Il broker Kafka è singolo, con fattore di replica uno: gli

acknowledgment richiesti non realizzano alta disponibilità. La tolleranza ai guasti non è il requisito individuale selezionato.

B. Completezza, carico e rappresentatività

QoS 0 e code limitate possono perdere telemetria. Il protocollo EOS certifica il completamento del flusso accettato dai componenti, non il recupero degli eventi persi a monte. Le run senza perdite devono quindi essere identificate tramite riconciliazione dei contatori e controlli sui risultati. Negli esperimenti di saturazione o rete degradata le perdite sono a loro volta un risultato da riportare; un throughput apparentemente stabile ottenuto scartando dati non soddisfa il requisito di scalabilità.

Il minimo dei watermark Edge evita risultati prematuri ma può trattenere finestre in memoria quando un Edge non avanza. Un timeout del runner interrompe una prova incompleta senza autorizzare il Global a ignorare la sorgente. Non sono implementati checkpoint, migrazione dello stato o una politica di esclusione dinamica degli Edge.

Il workload bilancia i sensori per zona e riguarda un solo tipo di dispositivo in un solo mese. Le medie sono pesate per numero di campioni validi: sensori più frequenti contribuiscono maggiormente. I risultati non possono essere generalizzati automaticamente ad altri dataset, algoritmi più costosi o installazioni geograficamente distribuite. Inoltre, la condivisione degli host e i limiti per container introducono contesa; la scalabilità delle repliche entro un host non dimostra un'illimitata capacità di espansione.

VII. CONCLUSIONI

L'applicazione realizza una pipeline Edge-Cloud in Go con validazione locale, aggregazione gerarchica e comunicazione persistente nei passaggi Kafka. La separazione fra partizioni logiche ed esecutori mantiene invariato il significato degli output al variare delle repliche. Watermark espliciti, progresso di partizione ed EOS permettono di concludere le finestre soltanto quando ogni contributo è determinato, includendo le sorgenti senza dati.

Le verifiche funzionali e gli artefatti preliminari forniscono una base per la valutazione, ma non consentono ancora di concludere che la scalabilità richiesta sia dimostrata. Il completamento del lavoro richiede una campagna EC2 valida e ripetuta, l'analisi dei tempi di disponibilità dei risultati e la valutazione di latenza e banda emulate. Le conclusioni quantitative dovranno essere aggiornate dopo tali esperimenti, conservando espliciti i limiti dello stato volatile e della telemetria con possibili perdite.

RIFERIMENTI BIBLIOGRAFICI

- [1] V. Cardellini, *Progetto A2: Applicazione Distribuita nel Compute Continuum*, Sistemi Distribuiti e Cloud Computing, Università degli Studi di Roma Tor Vergata, A.A. 2025/26, pp. 1–3.
- [2] Sensor.Community, *Get historical datas*, documentazione della comunità sull'archivio delle osservazioni. <https://forum.sensor.community/t/get-historical-datas/1648>. Consultato l'11 settembre 2026.
- [3] Apache Software Foundation, *Apache Kafka – Design*, documentazione 4.1. <https://kafka.apache.org/41/design/design/>. Consultato l'11 settembre 2026.
- [4] Apache Software Foundation, *Apache Avro 1.12.0 – Specification*. <https://avro.apache.org/docs/1.12.0/specification/>. Consultato l'11 settembre 2026.